

무궁화 밸런스 게임

Mugunghwa Balance Game — "무궁화꽃이 피었습니다" + 불안정한 집 운반 파티게임

NHN AI GAME HACKATHON 제출작 · v1 싱글플레이 프로토타입

문서 버전 2026-07-16 기준 (개발 진행에 따라 지속 최신화) | 팀 이성호, 김태우 | 기술 스택 TypeScript + Three.js + Vite (웹, 브라우저 실행)

1. 게임 개요

항목	내용
제목	무궁화 밸런스 게임 (Mugunghwa Balance Game)
한 줄 소개	"무궁화꽃이 피었습니다"의 멈춤 긴장감에 불안정한 집 나르기를 얹은 비대칭 경쟁 파티 게임
장르	캐주얼 / 파티 / 비대칭 경쟁 (술래 1 vs 무버 N)
플랫폼	웹 (브라우저 실행, 데스크톱 키보드)
플레이 인원	v1은 싱글플레이(무버 1 + 술래 1) 프로토타입, v2 멀티 확장 설계 반영
1회 플레이 시간	라운드당 약 90~120초

한눈에 보는 게임

플레이어(무버)는 등에 짐을 지고 결승선까지 걸어간다. 신호는 "무궁화 꽃이 피었습니다" 구호에 맞춰 **초록 (이동)** → **전환창** → **빨강(정지)**으로 바뀐다. 빨강일 때 움직이면 즉시 탈락. 그런데 이 게임의 핵심은 멈추는 순간에 있다 — 급하게 멈추면 등에 진 짐이 흔들려 떨어지고, 떨어진 짐은 그대로 내 점수 손실이자 술래의 점수가 된다.

2. 기획 의도 — 왜 이 소재인가

2.1 선택 이유 (진입 장벽 제로)

- 전 세계가 이미 아는 룰: 넷플릭스 <오징어 게임>의 "Red Light, Green Light"로 무궁화꽃이 피었습니다는 국적·연령 불문 누구나 안다. 심사자·플레이어에게 규칙 설명이 사실상 필요 없다.
- 즉시 이해 = 즉시 재미: "빨강에 멈춰라"는 한 문장이면 끝난다. 튜토리얼 없이 첫 판부터 플레이가 성립한다.
- 짧은 세션·반복성: 한 라운드가 90~120초로 짧아 파티/캐주얼 환경에 적합하고, 반복 플레이로 실력이 붙는다.

2.2 차별화 — "짐 = 점수" 한 줄이 만드는 딜레마

원작 룰이 쉬운 만큼 그냥 따라 만들면 밋밋하다. 그래서 불안정한 집 나르기를 얹어 원작에 없는 고민을 만들었다.

핵심 철학: 짐 = 점수. 한 번의 낙하가 "내 -1점 + 술래 +1점"의 자해(自害) 구조를 만든다.

- 짐은 곧 잠재 점수다. 많이 들고 완주할수록 점수가 높다.
- 하지만 짐이 많을수록 무겁고(느리고) 불안정하다 — **"탐욕이 곧 불안정"**.
- 급정지하면 짐이 떨어지고, 떨어진 짐은 사라지지 않고 트랙에 남아 **술래가 수확**한다. 내 손실이 곧 상대의 이득이 되는 제로섬 구조.

이 한 겹 덕분에 "빨리 갈까 vs 안전하게 갈까", "잡힐 위험을 감수할까 vs 짐을 흘릴까"라는 **매 순간의 선택**이 생긴다.

3. 핵심 재미 — 세 갈래의 딜레마

이 게임의 재미는 세 가지 트레이드오프가 겹쳐 있다는 데 있다.

(1) 속도 딜레마 — 빠름 vs 신중

- **빠름(속도 100%)**: 초록에서 많이 전진한다. 그러나 완전 정지까지 약 1.0초가 걸려, 전환창(0.8~1.2초)이 짧으면 빨강 전에 못 멈춰 **탈락 위험**. 급정지 시 짐이 크게 흔들린다.
- **신중(속도 50%)**: 느리지만 약 0.3초에 완전히 멈춘다. 전환창 안에서 항상 안전하게 정지 가능.

(2) 균형 딜레마 — 짐을 지킬까 vs 흘릴까

- 급정지는 스택(등짐)에 "킵"을 준다. 방치하면 기울기가 한쪽으로 쏠려 위쪽 짐부터 떨어진다.
- 좌우(A/D) 입력으로 되잡으면 낙하를 막을 수 있다. 단, **빨강 구간에서는 균형을 잡는 움직임도 경고로 누적**되어, 누적이 한계를 넘으면 잡힌다.

(3) 적재 딜레마 — 많이 들까 vs 가볍게 갈까

- 짐이 많으면 점수 상한이 높지만, 느려지고 불안정해져 흘릴 확률이 커진다.
- 짐이 적으면 빠르고 안정적이지만 얻을 점수가 낮다.

세 딜레마가 매 신호마다 동시에 걸리기 때문에, 쉬운 롤 위에서 **깊은 선택**이 만들어진다.

4. 게임 규칙

4.1 라운드 흐름 (상태머신)

LOBBY → GREEN(이동) → TELL(전환창) → RED(정지) → RESOLVE(정산) → ...반복... → END

페이지	구호 / 표시	의미	지속 시간
LOBBY	대기 중	시작 대기	—
GREEN	"무궁화 꽃이..."	이동 허용. 무버가 전진하는 유일한 구간	2.0~5.0초 (가변)
TELL	"무궁화 꽃이 피었..."	그린→레드 전환 예고 창. 여기서 멈춰야 안전	0.8~1.2초 (고정)
RED	"피었습니다!"	정지 필수. 움직이면 즉시 탈락	1.0~4.0초 (가변)
RESOLVE	정산	낙하·체포·점수 처리 창	0.6초
END	라운드 종료	시간 초과 또는 전원 완주/탈락	—

- **전환창(TELL)이 딜레마의 물리적 근거다.** 신중은 이 창 안에 완전 정지가 가능하지만, 빠름은 정지에 더 오래 걸려 급정지 위험이 생긴다. 이 창이 있기에 "속도 선택"이 성립한다.
- **종료 조건:** 라운드 최대 시간(90~120초) 경과, 또는 전원이 완주/탈락하면 즉시 END → 최종 정산.
- **빈 턴 반환:** RED 동안 아무 활동(이동/낙하/체포)이 없었다면 솔래가 "헛돈" 것으로 보고, 그 RED 시간을 라운드 예산에 되돌려준다.

4.2 조작

동작	키
전진 (누르는 동안 이동, 떼면 정지)	Space / W / ↑
좌우 균형 되잡기 (스택 기울기 보정)	A · D (또는 ← · →)
속도모드 토글 (빠름 100% ↔ 신중 50%)	Shift / E

전진은 1D 트랙(-z 축). 좌우 입력은 균형을 되잡기 위한 무게중심 이동. 조작은 생존 중(ALIVE)일 때만 반영된다.

4.3 이동·정지 모델 (Fake Physics)

- **최종 속도 = 기본속도(3.5) × 속도모드 배율 × 짐개수 배율**
- **짐개수 배율:** 짐 0개일 때 최대(1.4배) → 많을수록 선형 감속 → 하한(0.55배)에서 바닥. 과적하면 자기 제한적으로 느려진다.
- **정지 감속:** 빠름 약 1.0초, 신중 약 0.3초. 이 차이가 전환창 딜레마의 근거.

4.4 짐·균형·낙하 시스템 (스택 밸런스 킥)

- 무버 1명당 스칼라 **tilt(기울기)** 하나로 등짐 스택 전체를 표현(개별 강체 없음, 경량).

- **불안정성**: 짐이 많을수록($1 + 0.3 \times \text{짐수}$), 빠를수록, 결승선에 가까울수록 크게 쏠린다. 이동 중 방치하면 넘어가고, 정지하면 그 자세로 "얼어붙는다".
- **급정지 킥**: 전진 중 킥을 때의 순간 속력이 안전속도(1.2)를 넘으면 기울기에 임펄스. 빠를수록 큰 킥.
- **되잡기(A/D)**: 쏠린 쪽으로 입력해 counter-torque로 기울기를 0 근처로 되돌린다.
- **낙하 판정**: $|\text{tilt}|$ 가 낙하 임계(1.7)를 넘으면 초과량에 비례해 위쪽 짐부터 떨어진다. 낙하 후 스택이 낮아져 자기 보정.
- **아슬아슬(danger) 연출**: 위험 임계(1.0)에서 히스테리시스로 "어어어" 하며 가장자리에 머무는 긴장 연출.

4.5 낙하물 잔류 · 회수 시스템

- 떨어진 짐은 소멸하지 않고 트랙에 **DROPPED 상태로 남는다**(오브젝트 풀 재사용).
- **회수 규칙**: "자기보다 뒤에 있는 무버만" 반경(0.9) 내 낙하물 자동 흡수. 자기 낙하물은 재흡수 불가(급정지 직후 공짜 리필 방지).
- **재수확 상한**: 짐 1개당 술래가 가져가는 점수는 최대 3회(무한 점수 방지 다이얼).
- v1 싱글에서는 뒤 무버가 없어 회수 경로가 사실상 미검증 — 멀티/더미 도입 시 실측 필요.

4.6 RED 딜레마 (핵심 선택 지점)

- **W 전진(속도 임계 초과)** → 즉시 탈락.
- **A/D로 균형 보정** → 낙하는 막지만 **경고 +1**. 누적 경고가 한계(3)를 넘으면 탈락.
- **아무것도 안 함** → 짐을 흘려 점수를 잃지만 **생존은 유지**.

즉 "잡히거나(움직여서) vs 흘리거나(가만히)"의 선택이 빨강마다 반복된다.

5. 점수 시스템

복극성(설계 목표): 기대 술래 점수 $\approx$ 기대 1등 무버 점수. 어느 한쪽이 일방적으로 유리하지 않도록 수치를 조율한다.

5.1 무버 점수

무버 점수 = 반입 짐 수 $\times$ 1 + 생존보너스($K / \text{생존자 수}$)

완주 시 실제로 들고 들어간 짐만 점수가 된다. **탈락(CAUGHT) 무버는 0점**. 생존보너스는 생존자가 적을수록 커지는 후반 가속형(현재 $K = 28$).

5.2 술래 점수

술래 점수 = 낙하 수확 수 $\times$ 0.6 + 잡기보너스($c(c+1)/2$)

무버가 흘린 짐을 수확할수록, 무버를 많이 잡을수록 점수가 오른다. 잡기보너스는 c 번째로 잡으면 c 점씩 누적되는 후반 가속형.

5.3 밸런스 검증

코드에 점수 공식과 정합성 시뮬레이션(`simulateReference()`)이 내장되어 시작 시 콘솔에 1회 출력된다. **8인 레퍼런스 시나리오**(무버 7 + 술래 1, 술래 3잡·10낙하 수확, 생존 4, 1등 반입 5):

	튜닝 전	튜닝 후 ($K = 28$, 낙하배율 0.6)
술래 점수	16	12.0
1등 무버 점수	11	12.0
격차	5	0

콘솔에서 `__game.simulateReference({ seekerCatches: 5 })` 처럼 시나리오를 바꿔 즉석 실험 가능.

6. 현재 구현 현황 (2026-07-16 기준)

개발은 Phase 1~7까지 모두 완료되어 플레이 가능한 v1 프로토타입 상태다.

단계	내용	상태
Phase 1	셋업 + 고정 timestep 게임 루프 + 씬	완료
Phase 2	라운드 상태머신 + 게임플레이 데이터 모델	완료
Phase 3	입력 + 무버 이동 + 레드라이트 즉시 탈락 + 완주 판정	완료
Phase 4	급정지 짐 낙하 + 스택 밸런스 킥 + 트랙 잔류·회수 + 점수 귀속	완료
Phase 5	최종 점수 정산 + 라운드 종료 + 정합성 시물	완료
Phase 6	UI 오버레이 + 풀 기반 이펙트 + 3D 렌더	완료
Phase 7	폴리싱 + 밸런스 튜닝	완료

지금 플레이하면 되는 것

- 신호등 라운드 사이클(그린/텔/레드/정산) 자동 진행 + 페이지 배너·구호·색상.
- 빠름/신중 속도 토글, 전진, 좌우 균형 되잡기.
- 레드 중 이동 시 즉시 탈락, 경고 누적 탈락, 완주 판정.
- 급정지 → 스택 쏘림 → 짐 낙하 → 트랙 잔류 → (뒤 무버) 회수 → 점수 귀속.
- 우상단 UI: 타이머·내 짐·진행률·균형 게이지·경고·리더보드. 좌상단: 개발 디버그(fps/조작).
- 3D 캐릭터: KayKit glb 아바타 로드(실패 시 절차적 캐릭터로 자동 폴백), 걷기/idle 애니메이션, 화면 흔들림·플래시 이펙트, 카메라 추적.

아직 없는 것 (다음 목표)

- 실제 다중 무버(멀티/AI 더미) — v1은 무버 1 + 솔래 1.
- 라운드 재시작 UI(현재는 새로고침으로 리셋).
- 사운드/BGM.

7. 시스템 아키텍처 (기술 구조)

- **고정 timestep 루프**: `update(dt)` (결정론적 시뮬)와 `render(alpha)` (렌더) 분리, dt 상한으로 스파이럴 방지.
- **이벤트 기반 + 데이터 중심**: 상태관리 라이브러리 없이 자체 EventBus로 결합도를 낮춤. 이벤트 타입은 선언 병합으로 확장.
- **Fake physics**: 실제 강체 없음. 낙하·흔들림·술래 회전은 규칙과 트윈으로만 표현 → 가볍고 결정론적.
- **오브젝트 풀링**: 짐·이펙트·낙하물 표시를 전부 풀에서 재사용해 GC 부담 최소화.
- **수치 분리 원칙**: 모든 밸런스 값은 `src/config/` 에만 정의(하드코딩 금지) → 튜닝이 한곳에서.

```
src/  
  core/      GameLoop, EventBus, ObjectPool  
  config/    GameBalance, PlayerConfig, ItemConfig, BalanceConfig, EffectConfig,  
             ArtConfig, VisualConfig, AssetConfig  (모든 수치)  
  gameplay/  RoundStateMachine, RoundState, Mover, Seeker, Item, MovementSystem,  
             BalanceSystem, PlayerSystem, CargoSystem, ScoreSystem,  
             BalanceDebug, GameplayEvents, types  
  input/     InputController  
  render/    SceneManager, GameRenderer, MoverView, CharacterView,  
             SeekerView, DroppedItemsView, EffectSystem  
  ui/        UIOverlay  
  main.ts    진입점
```

8. 밸런스 수치표 (현재 튜닝값)

항목	값	위치	비고
속도 배율 (빠름/신중)	1.0 / 0.5	GameBalance.speed	확정 규칙
안정화 시간 (빠름/신중)	1.0s / 0.3s	GameBalance.stabilize	확정 규칙
GREEN 지속	2.0~5.0s	GameBalance.signal	튜닝 노브
TELL 전환창	0.8~1.2s	GameBalance.signal	확정(딜레마 근거)
RED 지속	1.0~4.0s	GameBalance.signal	확정 규칙
RESOLVE 지속	0.6s	GameBalance.signal	튜닝 노브
기본 이동 속도	3.5 유닛/s	PlayerConfig	
짐 속도배율 (0개/상한/하한)	1.4 / 0.7 / 0.55	PlayerConfig.itemSpeed	
시작 짐 / 상한	5 / 5	PlayerConfig · ItemConfig	확정 규칙
낙하 임계 (tilt)	1.7	BalanceConfig	Phase 4rev 상향
위험(danger) 임계	1.0	BalanceConfig	연출용
안전 속도 (킥 없음)	1.2	BalanceConfig	
경고 한계 / 디바운스	3회 / 0.35s	BalanceConfig	RED 균형보정 누적
회수 반경	0.9	ItemConfig	
재수확 상한	3	ItemConfig	무한 faucet 방지
생존보너스 K	28	GameBalance.score	Phase 7: 24→28 (무버 ↑)
낙하 점수 배율	0.6	GameBalance.score	Phase 7: 1→0.6 (술래 ↓)
잡기 보너스 배율	1	GameBalance.score	
트랙 길이	32 유닛	GameBalance.track	튜닝 노브
이동 감지 임계	0.08	GameBalance.judge	RED 정지 판정

9. 팀 구성 및 역할

팀원	담당 역할
이성호	작성 예정 — 확인 후 채움
김태우	작성 예정 — 확인 후 채움

이 섹션은 두 팀원의 실제 담당 영역(기획·개발·아트·AI 활용 등)을 확인한 뒤 구체적으로 채웁니다.

10. 남은 과제 · 로드맵

밸런스/설계 미결

- 회수 스캐빈저 남용 방지: 뒤 무버가 앞 낙하물을 공짜로 쓸어담는 악용에 대한 비용/딜레이/횟수 제한 정책 미정.
- 재수확 상한(3)의 적정성 실험/플레이 미검증.
- 멀티/더미 무버 부재: v1 싱글이라 회수·비대칭 경쟁이 실측되지 않음.

기능 확장

- 다중 무버(로컬 멀티 또는 AI 더미), 라운드 재시작 UI, 사운드/BGM.
- 저사양 기기 프레임 프로파일링 후 이펙트 풀 크기·빈도 조정.

제출물 연동 (NHN AI GAME)

- 플레이 가능한 웹 빌드(GitHub Pages) + 소스 공개.
- 30~60초 플레이 영상(YouTube).
- 게임 소개·AI 활용 기술·팀원 롤 문서(PDF) — 본 기획서를 기반으로 파생.

11. 외부 에셋 · 출처

에셋	용도	출처 / 라이선스
KayKit Adventurers 2.0 (FREE)	3D 캐릭터 메시·애니메이션 (Knight/Mage/Ranger/Rogue/Barbarian 등, Rig_Medium)	KayKit (Kay Lousberg) — 무료 배포 판. 최종 제출 전 라이선스 조건 재확인 필요
Three.js (r0.164)	3D 렌더링 엔진	MIT
Vite / TypeScript	빌드·개발 환경	MIT / Apache-2.0

외부 에셋의 정확한 라이선스 문구·저작자 표기는 제출용 AI 활용 기술 문서에 최종 정리합니다.

본 문서는 프로젝트 진행에 따라 지속적으로 최신화됩니다.